

Question: 01

Which part of the serial code can be paralyzed?

Answer:

We can paralyze both the for loops in which different computation is being done.

- I. Arr[i] = I * 0.5;**
- II. Sum += arr[i];**

We can achieve the maximum speed up by paralyzing these loops

Justification:

As you can see the code contains the value of N that is too huge to be computed with less time, also there are two loops running on that value of N so it becomes even more time consuming. We can paralyze these loops to get the maximum speedup.

Question: 02

Paralyze the selected part using correct construct. Show the answer of the serial code and the parallel version of the code.

Serial code:

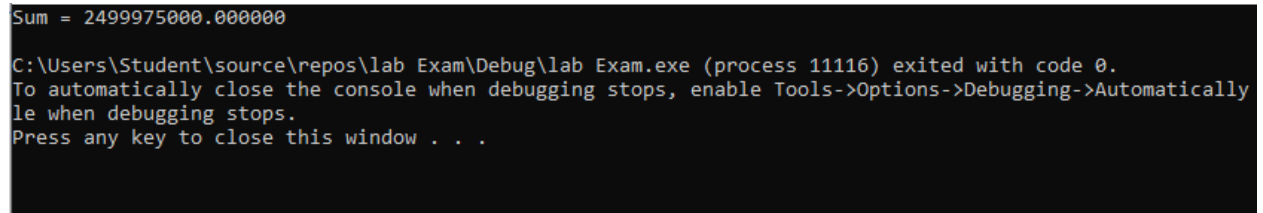
```
#include<stdio.h>

#include<omp.h>

int main()
{
    int n = 100000;
    double sum = 0.0;
    double arr[100000];
    for (int i = 0; i < n; i++)
        arr[i] = i * 0.5;
    for (int i = 0; i < n; i++)
```

```
        sum += arr[i];  
  
    printf("Sum = %f\n", sum);  
  
    return 0;  
  
}
```

Output of the Serial Version:



```
Sum = 2499975000.000000  
  
C:\Users\Student\source\repos\lab Exam\Debug\lab Exam.exe (process 11116) exited with code 0.  
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically  
close when debugging stops.  
Press any key to close this window . . .
```

Paralyzed code:

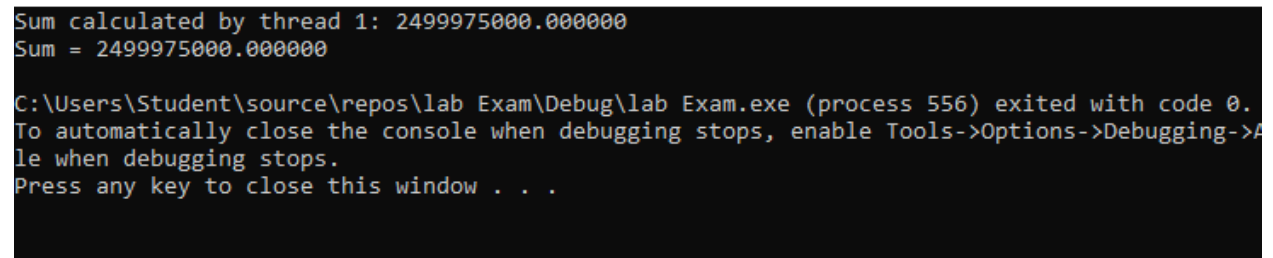
```
int main()  
{  
    int n = 100000;  
    double sum = 0.0;  
    double arr[100000];  
  
    #pragma omp parallel  
    {  
        #pragma omp sections  
        {  
            #pragma omp section  
            {  
                for (int i = 0; i < n; i++)  
                    arr[i] = i * 0.5;  
            }  
        }  
    }  
}
```

```
#pragma omp section
{
    for (int i = 0; i < n; i++)
        sum += arr[i];

    printf("Sum calculated by thread %d: %f\n", omp_get_thread_num(),
sum);
}
}

printf("Sum = %f\n", sum);
return 0;
}
```

Output:



```
Sum calculated by thread 1: 2499975000.000000
Sum = 2499975000.000000

C:\Users\Student\source\repos\lab Exam\Debug\lab Exam.exe (process 556) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Auto
le when debugging stops.
Press any key to close this window . . .
```

Task: 02

Question: 01

Please use pragma omp barrier at the correct place.

Answer:

Serial code:

```
#include<stdio.h>
#include<omp.h>
#include<math.h>
#define N 4
int main()
{
    double A[N][N] = {
        {1,2,3,4},
        {2,3,4,5},
        {3,4,5,6},
        {4,5,6,7}
    };
    for (int i = 0; i < N; i++)
    {
        double sum = 0.0;
        for (int j = 0; j < N; j++)
            sum += A[i][j] * A[i][j];
        printf("sum = %f\n", sum);

        double norm = sqrt(sum);
        for (int j = 0; j < N; j++)
            A[i][j] /= norm;
    }
}
```

```
printf("Matrix normalized\n");

return 0;

}
```

Output of serial:

```
sum = 30.000000
sum = 54.000000
sum = 86.000000
sum = 126.000000
Matrix normalized
C:\Users\Student\source\repos\labquiz\Debug\labquiz.exe (process 11840) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->A
le when debugging stops.
Press any key to close this window . . .
```

Paralyzed code:

```
#include<stdio.h>

#include<omp.h>

#include<math.h>

#define N 4

int main()

{

    double A[N][N] = {

        {1,2,3,4},

        {2,3,4,5},

        {3,4,5,6},

        {4,5,6,7}

    };

    #pragma omp parallel for
```

```
for (int i = 0; i < N; i++)
{
    int tid = omp_get_thread_num();

    double sum = 0.0;
    for (int j = 0; j < N; j++)
        sum += A[i][j] * A[i][j];
    printf("Thread %d computed sum for row %d = %f\n", tid, i, sum);

    double norm = sqrt(sum);

#pragma omp barrier

    for (int j = 0; j < N; j++)
        A[i][j] /= norm;
    printf("Thread %d normalized row %d\n ", tid, i);
}

printf("Matrix normalized\n");
for (int i = 0; i < N; i++)
{
    for (int j = 0; j < N; j++)
        printf("%f ", A[i][j]);
```

```
        printf("\n");  
    }  
  
    return 0;  
}
```

Output:

```
Thread 0 computed sum for row 0 = 30.000000  
Thread 0 normalized row 0  
Thread 0 computed sum for row 1 = 54.000000  
Thread 0 normalized row 1  
Thread 0 computed sum for row 2 = 86.000000  
Thread 0 normalized row 2  
Thread 0 computed sum for row 3 = 126.000000  
Thread 0 normalized row 3  
Matrix normalized  
0.182574 0.365148 0.547723 0.730297  
0.272166 0.408248 0.544331 0.680414  
0.323498 0.431331 0.539164 0.646997  
0.356348 0.445435 0.534522 0.623610
```

Task: 03

Code:

```
#include<stdio.h>  
  
#include<math.h>  
  
int main()  
{  
    int n = 1000;  
    int arr[1000];  
    for (int i = 0; i < n; i++)  
        arr[i] = 100 % 10000;
```

```
int max_val = arr[0];  
for (int i = 1; i < n; i++)  
{  
    if (arr[i] > max_val)  
        max_val = arr[i];  
}  
  
printf("Maximum = %d\n", max_val);  
return 0;  
  
}
```